Week 2 - Amazon ECS and Fargate (rolling vs CodeDeploy blue/green, task definitions, circuit breaker)

DOP-C02 Domain 1 (SDLC Automation). Exam focus: task definition settings and the two IAM roles, rolling update percentages, the deployment circuit breaker and alarm rollback, CodeDeploy blue/green for ECS, and the CodePipeline ECS deploy actions.

Part 1 - Building blocks

Term	Meaning
Cluster	A logical group of tasks and services, plus the capacity they run on
Task definition	A JSON blueprint for one or more containers: image, CPU and memory, ports, roles, logs, secrets. It has **revisions**
Task	A running copy of a task definition
Service	Keeps a **desired count** of tasks running, replaces failed tasks, registers them with a load balancer, and handles **deployments**
Container instance	An EC2 instance running the ECS agent (EC2 launch type only)
Capacity provider	Where tasks run: **FARGATE**, **FARGATE_SPOT**, or an **EC2 Auto Scaling group** capacity provider

Launch types and capacity

- **Fargate:** serverless. No instances to patch or scale. You pay per task for vCPU, memory and storage. Every task gets its own ENI (`awsvpc`).
- **EC2:** you manage the instances, with more control (GPUs, privileged containers, daemon tasks, any instance type).
  - **ASG capacity providers** support **managed scaling** (the ASG scales to fit the tasks) and **managed termination protection** (instances running tasks aren't terminated during scale-in).
- **Fargate Spot:** up to about 70% cheaper. Tasks can be stopped with a **2-minute warning** (SIGTERM). For fault-tolerant work.
  - A **capacity provider strategy** with **base** and **weight** mixes FARGATE and FARGATE_SPOT, for example base 2 on FARGATE, then 1:3.
- **ECS Anywhere:** run tasks on your own servers.

Part 2 - Task definitions

Revisions

- A task definition is identified as **family:revision**, for example `orders:12`.
- `RegisterTaskDefinition` always creates a **new revision**. Existing revisions are **immutable**.
- A deployment means updating the **service** to use a new revision.
- Old revisions can be **deregistered** (INACTIVE: running tasks continue, no new ones start) and then **deleted**.

Task-level settings

Setting	Notes
`family`	The name
`requiresCompatibilities`	`FARGATE`, `EC2` or `EXTERNAL`
`networkMode`	`awsvpc` (required on Fargate: own ENI, private IP and **security groups**), `bridge`, `host` or `none` (EC2 only)
`cpu` / `memory`	**Required on Fargate**, in fixed combinations (for example 0.25 vCPU with 0.5 to 2 GB, up to 16 vCPU with 120 GB)
`executionRoleArn`	Used by the **ECS agent / Fargate** to **pull images from ECR**, **send logs**, and **fetch secrets** at startup
`taskRoleArn`	Used by your **application code** to call AWS APIs (S3, DynamoDB, SQS)
`volumes`	Bind mounts, Docker volumes (EC2), **EFS** (EC2 and Fargate). Fargate ephemeral storage **20 GiB default, up to 200 GiB**
`runtimePlatform`	`X86_64` or `ARM64` (Graviton), Linux or Windows
`placementConstraints`	EC2 only

- **Biggest exam point:** the app gets AccessDenied calling S3 means fix the **task role**. The task can't pull its image, write logs or read a secret at startup means fix the **execution role**.

Container definitions

Setting	Notes
`name`, `image`	Image by **tag** (`repo:1.4.2`) or **digest** (`repo@sha256:...`)
`memory` / `memoryReservation`	`memory` is a **hard limit**: over it the container is killed (**exit 137, OutOfMemory**). `memoryReservation` is a **soft limit**
`essential`	If an **essential** container stops, the **whole task stops**. Sidecars often use `essential: false`
`portMappings`	With `awsvpc`, host port equals container port. With `bridge`, host port 0 gives **dynamic port mapping** behind an ALB
`environment` / `environmentFiles`	Plain variables, or a `.env` file in **S3**
`secrets`	`name` plus `valueFrom` = ARN of an **SSM parameter** or **Secrets Manager** secret. Injected **at task start**
`logConfiguration`	`awslogs` (CloudWatch Logs), `awsfirelens` (Fluent Bit sidecar to S3, OpenSearch, Firehose, third parties), `splunk`
`healthCheck`	Container health check: command, interval, timeout, retries, startPeriod
`dependsOn`	Start order: `START`, `COMPLETE`, `SUCCESS` or `HEALTHY` (sidecars and init containers)
`privileged`, `readonlyRootFilesystem`	Security settings. `privileged` **isn't allowed on Fargate**
`stopTimeout`	Time between SIGTERM and SIGKILL, up to **120 seconds on Fargate** (graceful shutdown, Spot)

Secrets

- The **execution role** needs `ssm:GetParameters` or `secretsmanager:GetSecretValue`, plus `kms:Decrypt` for a customer managed key.
- Secrets are read **only when the task starts**. After **rotation**, running tasks keep the old value. **Force a new deployment** (`aws ecs update-service --force-new-deployment`), or have the app read the secret at runtime with the task role.
- **Never** put secrets in plain `environment` values.

Images: tags vs digests

- **Mutable tags** like `:latest` are risky. ECS now **resolves the tag to a digest at deployment**, so every task in one deployment runs the **same image**.
- To pick up a new image pushed under the **same tag**, use `--force-new-deployment`.
- **Best practice:** a unique tag per build (commit SHA) or a digest, a **new task definition revision** per release, and **ECR tag immutability**.

Pulling from private ECR

- The **execution role** needs `ecr:GetAuthorizationToken`, `ecr:BatchGetImage`, `ecr:GetDownloadUrlForLayer`. The managed policy `AmazonECSTaskExecutionRolePolicy` covers ECR and logs.
- In **private subnets**: a **NAT gateway**, or **VPC endpoints** for `ecr.api`, `ecr.dkr`, **S3 (gateway endpoint, image layers)**, `logs`, and `ssm` / `secretsmanager` if secrets are used.
- In public subnets on Fargate, **assign a public IP**, or the task can't reach ECR.

Error	Usual cause
`CannotPullContainerError`	Missing execution role permissions, no route to ECR (NAT or endpoints), wrong image or tag, cross-account repository policy missing
`ResourceInitializationError`	Couldn't fetch secrets or set up logging at startup: execution role permissions or no route to SSM, Secrets Manager or Logs
`OutOfMemoryError` / exit 137	The container went over its hard `memory` limit
Essential container in task exited	The main app crashed. Check its logs

- **Common sidecars:** FireLens (logs), X-Ray daemon (tracing), CloudWatch agent (metrics), Envoy (App Mesh / Service Connect). Use `dependsOn` so the app starts after the sidecar is ready.

Part 3 - Services and deployment controllers

deploymentController	Who deploys	Pattern
ECS (default)	ECS itself	**Rolling update**. Newer versions also have built-in blue/green, linear and canary strategies
CODE_DEPLOY	**CodeDeploy**	**Blue/green** with traffic shifting: all at once, canary or linear
EXTERNAL	A third-party tool using the task set APIs	Custom (Jenkins, Spinnaker)

Part 4 - Rolling updates (ECS controller)

Parameter	Meaning	Default
`minimumHealthyPercent`	The **lowest** number of tasks that must stay RUNNING and healthy, as a percentage of desired count (rounded **up**)	**100%**
`maximumPercent`	The **highest** number of tasks allowed (old plus new), as a percentage of desired count (rounded **down**)	**200%**

Worked examples (desired count = 4)

min / max	What happens	Trade-off
100 / 200	Starts **4 new** tasks (8 total), waits until healthy, then stops the 4 old	Full capacity, fastest. Needs room for double the tasks
50 / 100	Stops **2 old** first, starts 2 new, then the next 2	**No extra capacity**, but capacity drops to 50%
100 / 150	Starts 2 new (6 total), stops 2 old, repeats	Full capacity with less extra room
0 / 100	Stops all tasks, then starts new ones	**Downtime**, like all at once
100 / 100	Can't stop or add any task, so **the deployment can't proceed**	A classic trap

Other rolling update settings and behaviour

- `healthCheckGracePeriodSeconds`**:** how long the service **ignores ELB health check failures** after a task starts. Set it longer than the app's start-up time, or slow-starting tasks are killed again and again.
- **Health signals:** the **ELB target group health check**, the **container** `healthCheck`, and Cloud Map health.
- **ALB deregistration delay** (connection draining): old tasks finish in-flight requests before stopping. Lowering it from 300 seconds speeds up deployments when requests are short.
- **States:** the new deployment is **PRIMARY**, the old one **ACTIVE**, then INACTIVE. `rolloutState` is `IN_PROGRESS`, `COMPLETED` or `FAILED`.
- **Daemon services** (one task per EC2 instance) use only `minimumHealthyPercent`. No daemon scheduling on Fargate.
- **Without a circuit breaker**, a broken task definition makes ECS **keep retrying forever** and the deployment seems stuck.
- **CloudFormation:** changing the task definition of an `AWS::ECS::Service` triggers a rolling deployment, and CloudFormation **waits for the service to become stable**. A circuit breaker makes a bad deployment fail fast so the stack rolls back.

Part 5 - Deployment circuit breaker and CloudWatch alarm rollback

```
"deploymentConfiguration": {
  "minimumHealthyPercent": 100,
  "maximumPercent": 200,
  "deploymentCircuitBreaker": { "enable": true, "rollback": true },
  "alarms": { "alarmNames": ["orders-5xx-high", "orders-latency-high"], "enable": true, "rollback": true }
}
```

Circuit breaker

- Works **only with the ECS (rolling) deployment controller**. **Not available with CODE_DEPLOY**, which uses CodeDeploy's own automatic rollback.
- Two stages:
  1. Tasks **fail to reach RUNNING** (image pull errors, crashes, missing secrets).
  2. Tasks reach RUNNING but **fail health checks** (ELB, container health check or Cloud Map).
- **Failure threshold:** failed tasks equal to **half the desired count, at least 3 and at most 200**. The deployment is then **FAILED**.
- `rollback: true`**:** ECS rolls back to the **last deployment that COMPLETED**. `rollback: false`**:** the deployment stops as FAILED and old tasks keep serving.
- ECS sends **EventBridge "ECS Deployment State Change"** events (`SERVICE_DEPLOYMENT_IN_PROGRESS`, `SERVICE_DEPLOYMENT_COMPLETED`, `SERVICE_DEPLOYMENT_FAILED`) for notifications and automation.

CloudWatch alarm-based rollback

- `deploymentConfiguration.alarms` makes ECS **watch CloudWatch alarms** (ALB 5XX count, latency, business metrics) **during the deployment**.
- An alarm in ALARM fails the deployment and rolls back if `rollback` is true.
- It catches **code that is healthy but wrong**, which the circuit breaker can't see. Use **both together**.

Problem	Caught by
Image can't be pulled, container crashes, secret missing	**Circuit breaker** (stage 1)
Tasks start but fail the ALB health check	**Circuit breaker** (stage 2)
Tasks healthy but 5XX rate or latency rises	**CloudWatch alarm rollback**
Errors during a CodeDeploy blue/green deployment	**CodeDeploy deployment group alarms and automatic rollback**

Part 6 - CodeDeploy blue/green for ECS (CODE_DEPLOY controller)

What you need

1. An ECS service created with `deploymentController: CODE_DEPLOY`. The controller **can't be changed later**; create a new service.
2. An **ALB or NLB** with **two target groups** (blue and green), a **production listener** (for example 443), and an optional **test listener** (for example 8443) to test green before users reach it.
3. A CodeDeploy **application** (compute platform **ECS**) and a **deployment group**: cluster, service, load balancer, both target groups, listeners, service role (`AWSCodeDeployRoleForECS`), alarms and rollback settings.

How a deployment works

1. CodeDeploy creates a **replacement task set (green)** with the new task definition and registers it to the **green target group**.
2. The **test listener** sends test traffic to green. Hooks can run tests here.
3. **Production traffic shifts** from blue to green, using the deployment configuration.
4. The **original task set (blue)** is kept for the **termination wait time** (`terminationWaitTimeInMinutes`, up to **2,880 minutes = 2 days**), then terminated. Until then, **rollback is instant**.

Deployment configurations (ECS)

Configuration	Behaviour
`CodeDeployDefault.ECSAllAtOnce`	100% at once (the only option with an **NLB**)
`CodeDeployDefault.ECSLinear10PercentEvery1Minutes`	+10% every minute
`CodeDeployDefault.ECSLinear10PercentEvery3Minutes`	+10% every 3 minutes
`CodeDeployDefault.ECSCanary10Percent5Minutes`	10% for 5 minutes, then 100%
`CodeDeployDefault.ECSCanary10Percent15Minutes`	10% for 15 minutes, then 100%
Custom	Your own canary or linear percentages and intervals

Lifecycle hooks (Lambda functions)

Hook	When	Typical use
`BeforeInstall`	Before the green task set is created	Setup checks
`AfterInstall`	Green created, no traffic yet	Smoke tests directly on the tasks
`AfterAllowTestTraffic`	Test listener points to green	**Integration tests through the test listener**
`BeforeAllowTraffic`	Before production traffic shifts	Final checks
`AfterAllowTraffic`	After production traffic has fully shifted	Post-deployment checks

- Each hook must call `PutLifecycleEventHookExecutionStatus` (Succeeded or Failed). Hook timeout is **1 hour**.
- A **Failed** hook or a deployment group **alarm** gives an **automatic rollback** to blue.

ECS appspec

```
version: 0.0
Resources:
  - TargetService:
      Type: AWS::ECS::Service
      Properties:
        TaskDefinition: "arn:aws:ecs:us-east-1:111122223333:task-definition/orders:13"
        LoadBalancerInfo:
          ContainerName: "web"
          ContainerPort: 8080
        PlatformVersion: "LATEST"
Hooks:
  - AfterAllowTestTraffic: "CodeDeployHook_orders_IntegrationTests"
  - BeforeAllowTraffic: "CodeDeployHook_orders_FinalCheck"
```

- In CodePipeline the task definition is usually the placeholder `<TASK_DEFINITION>`.

Limits with CODE_DEPLOY

- You **can't change the task definition, load balancer or network settings through** `UpdateService`. New versions go through a **CodeDeploy deployment**. Desired count and grace period can still be changed.
- The **circuit breaker and ECS alarm rollback don't apply**. Use the deployment group's alarms and automatic rollback.
- Double the tasks run during the shift, so it costs more than a rolling update.

Newer: built-in ECS blue/green, linear and canary

- Since 2025 the **ECS** controller can do **blue/green** (strategy `BLUE_GREEN`) with a **bake time**, **lifecycle hooks** (Lambda) at stages such as test and production traffic shift, ALB, NLB and **Service Connect**, plus the circuit breaker and alarms. **Linear** and **canary** strategies followed.
- Most exam questions still expect **CodeDeploy** for ECS blue/green, canary and linear.

Part 7 - ECS in CodePipeline

	Rolling update	Blue/green
Deploy action	**Amazon ECS** (provider `ECS`)	**Amazon ECS (Blue/Green)** (provider `CodeDeployToECS`)
Build output file	`imagedefinitions.json`	`imageDetail.json`
Format	A list of container name plus imageUri	One ImageURI
Other inputs	None. ECS copies the current task definition, swaps the image, registers a new revision and updates the service	`taskdef.json` (image placeholder such as `<IMAGE1_NAME>`) and `appspec.yaml` (`<TASK_DEFINITION>` placeholder)
Who shifts traffic	ECS (rolling, circuit breaker)	CodeDeploy (blue/green, hooks, alarms)

```
# buildspec for the rolling ECS deploy action
post_build:
  commands:
    - docker push $REPO_URI:$IMAGE_TAG
    - printf '[{"name":"web","imageUri":"%s"}]' $REPO_URI:$IMAGE_TAG > imagedefinitions.json
artifacts:
  files: [imagedefinitions.json]
```

- **Traps:** the rolling action fails if `imagedefinitions.json` is missing or the **container name doesn't match** the task definition. The blue/green action needs `taskdef.json` plus `appspec.yaml` (plus `imageDetail.json` if the image comes from the ECR source or build).
- **CloudFormation route:** `Transform: AWS::CodeDeployBlueGreen` with the `AWS::CodeDeploy::BlueGreen` **hook** does ECS blue/green through stack updates.

Part 8 - Rolling vs CodeDeploy blue/green

	Rolling (ECS)	Blue/green (CodeDeploy)
Mechanism	Replaces tasks within min% / max%	New task set behind a second target group, then a listener switch
Load balancer	Optional	**Required** (ALB or NLB)
Mixed versions serving	**Yes**, during the rollout	Only during a canary or linear shift
Test before production traffic	No	**Yes**, test listener and `AfterAllowTestTraffic`
Canary or linear	No (unless the newer built-in strategies)	**Yes**
Rollback	Circuit breaker or alarms redeploy the old revision (takes time)	**Instant**: reroute to blue while it still runs
Failure detection	Circuit breaker plus CloudWatch alarms	Deployment group alarms plus hooks
Extra capacity	Set by `maximumPercent` (can be 100% for none)	Double the tasks
Complexity and cost	Low	Higher

How to read exam questions

The question says	Answer
Roll back automatically if new tasks fail to start or fail health checks, simplest	**Rolling + circuit breaker with rollback**
Roll back if the 5XX rate rises during a rolling deployment	**deploymentConfiguration alarms** with rollback
Test with a test listener before production traffic	**CodeDeploy blue/green + AfterAllowTestTraffic**
Shift 10% for 5 minutes, then the rest	**CodeDeploy ECSCanary10Percent5Minutes**
Instant rollback, keep the old version for 1 hour	**Blue/green with termination wait time**
Deploy without any extra capacity	**Rolling with min 50 / max 100**
Stuck: desired 1, min 100, max 100	Raise **maximumPercent** or lower **minimumHealthyPercent**
New tasks killed because the app starts slowly	Increase `healthCheckGracePeriodSeconds`
App can't call DynamoDB	**Task role**
Task can't pull from ECR or read secrets	**Execution role** plus a network route (NAT or VPC endpoints)
Pick up a rotated secret or a new image with the same tag	`--force-new-deployment`

Part 9 - Fargate details, scaling and monitoring

Fargate specifics

- **Platform versions:** `LATEST` or a specific version (for example 1.4.0). AWS patches the infrastructure.
- **Task retirement:** for maintenance AWS **retires** tasks and the service replaces them. AWS Health notifies you. The account setting `fargateTaskRetirementWaitPeriod` can be 0, 7 or 14 days.
- **Not on Fargate:** privileged containers, `host` / `bridge` network modes, daemon services, placement strategies, and (on most configurations) GPUs.
- **ECS Exec:** `aws ecs execute-command` opens a shell in a running container. Needs `enableExecuteCommand` and SSM permissions in the **task role**. Sessions can be logged to CloudWatch or S3.
- **Placement** (EC2 only): strategies `binpack` (cost), `spread` (availability, for example by AZ), `random`; constraints `distinctInstance` and `memberOf`.

Service Auto Scaling

- **Target tracking** on `ECSServiceAverageCPUUtilization`, `ECSServiceAverageMemoryUtilization` or `ALBRequestCountPerTarget`.
- **Step scaling** (for example on SQS queue depth for workers) and **scheduled scaling**.
- On EC2, capacity provider **managed scaling** adds instances when tasks can't be placed.

Monitoring and troubleshooting

- **Container Insights** (with enhanced observability): CPU, memory, network and restarts per cluster, service and task.
- **Service events**, for example unable to place a task, or reached a steady state.
- **Stopped tasks:** `stoppedReason` and each container's exit code.
- **EventBridge:** Task State Change, Service Action and Deployment State Change events.
- **ALB problems:** health check path or port mismatch, **task security group not allowing the ALB security group**, and target type must be `ip` for `awsvpc` / Fargate.

Quick memory summary

- **Task definition** = `family:revision`, immutable revisions. **Execution role** = pull, logs, secrets at start. **Task role** = the app's AWS calls. Fargate needs `awsvpc` and task-level CPU and memory. Secrets are read at **start only**: **force a new deployment** after rotation.
- **Rolling:** `minimumHealthyPercent` (default 100) and `maximumPercent` (default 200). 100/200 full capacity, 50/100 no extra capacity, 100/100 stuck. Grace period for slow starts.
- **Circuit breaker** (rolling only): fails at half the desired count of failed tasks (min 3, max 200); with `rollback`, back to the last COMPLETED deployment. Add **CloudWatch alarms** for healthy-but-bad code.
- **CodeDeploy blue/green** (`CODE_DEPLOY`, fixed at creation): ALB or NLB, **two target groups**, production and **test listener**, replacement task set, AllAtOnce / Canary / Linear (**NLB = AllAtOnce only**), hooks BeforeInstall, AfterInstall, **AfterAllowTestTraffic**, BeforeAllowTraffic, AfterAllowTraffic, termination wait up to **2 days**.
- **CodePipeline:** rolling = `imagedefinitions.json`; blue/green = `imageDetail.json` **+** `taskdef.json` **+** `appspec.yaml`.
- **Newer:** ECS built-in blue/green, linear and canary (2025). CodeDeploy remains the classic exam answer.